

Deep Learning

3b. Entraîner un modèle avec torch

Romain Tavenard

Les 4 ingrédients d'un entraînement

- **Un modèle** : une fonction paramétrée $m(x; \theta)$
→ `torch.nn.Module`
- **Une fonction de perte** : mesure de l'erreur $\mathcal{L}$
→ `torch.nn.MSELoss`, `torch.nn.CrossEntropyLoss`, ...
- **Des données**, servies par *mini-batches*
→ `torch.utils.data.Dataset` + `DataLoader`
- **Un optimiseur** : applique les mises à jour de θ
→ `torch.optim.SGD`, `torch.optim.Adam`, ...

Définir un modèle : nn.Module

```
import torch
import torch.nn as nn

class MLP(nn.Module):
    def __init__(self, n_in, n_hidden, n_out):
        super().__init__()
        self.fc1 = nn.Linear(n_in, n_hidden)
        self.fc2 = nn.Linear(n_hidden, n_out)

    def forward(self, x):
        h = torch.relu(self.fc1(x))
        return self.fc2(h)    # pas d'activation de sortie ici

model = MLP(n_in=13, n_hidden=64, n_out=1)
y_hat = model(x)            # appelle model.forward(x)
```

- `__init__` : **déclarer** les couches (et donc les paramètres)
- `forward` : **décrire le calcul** de la sortie à partir de l'entrée

Fonctions de perte en torch

- Une perte est aussi un `nn.Module` : on l'instancie, puis on l'appelle

```
loss_fn = nn.MSELoss()           # régression
loss = loss_fn(y_hat, y)         # scalaire (moyenne sur le batch)
```

- **Classification** : `nn.CrossEntropyLoss`

```
loss_fn = nn.CrossEntropyLoss()
logits = model(x)                # (batch, n_classes), SANS softmax
loss = loss_fn(logits, y)        # y : indices de classes (batch,)
```

- ▶  le softmax est **inclus** dans la perte :
pas de softmax en sortie du modèle
- ▶ cas binaire : `nn.BCEWithLogitsLoss` (sigmoïde incluse)

Dataset et DataLoader

- Dataset : **accès à un exemple** (x_i , y_i) par son indice

```
from torch.utils.data import TensorDataset, DataLoader

X = torch.tensor(X_np, dtype=torch.float32)
y = torch.tensor(y_np, dtype=torch.float32)
dataset = TensorDataset(X, y)      # dataset[i] -> (X[i], y[i])
```

- DataLoader : **découpage en *mini-batches***

```
loader = DataLoader(dataset, batch_size=32, shuffle=True)

for x_batch, y_batch in loader:    # un tour de boucle = un mini-batch
    ...                            # un tour complet = une epoch
```

- ▶ `shuffle=True` : nouvel ordre à chaque *epoch* (SGD, cf. séance 2)
- ▶ pour la validation / le test : `shuffle=False`

Conclusion

`nn.Module`

architecture : couches dans `__init__`, calcul dans `forward`

Perte

un `nn.Module` qui compare prédictions et cibles
(⚠ `CrossEntropyLoss` attend des *logits*)

`DataLoader`

itère sur les données par *mini-batches* mélangés

Optimiseur

`zero_grad()` → `backward()` → `step()`